

Software Requirements Document (SRD)

Project Name

OneConfig

Document Control

- Version: 0.1
- Date: May 2026
- Status: Draft

1. Purpose

OneConfig is a developer tool that lets a repository define its local setup in a single config file, then spin up a working development environment with one command. The goal is to remove the friction of onboarding, dependency installation, and local service setup.

2. Problem Statement

Developers waste time reading READMEs, installing mismatched tool versions, setting environment variables, and starting dependent services manually. Existing solutions are often fragmented across multiple language managers, docker files, scripts, or complex infrastructure tools. OneConfig solves this by making local setup predictable, repeatable, and simple.

3. Target Users

- Open-source contributors.
- Small engineering teams.
- New hires onboarding to a codebase.
- Solo developers who want reproducible local environments.

4. Product Vision

A user clones a repository, runs one command, and gets a ready-to-code local environment without reading a long setup guide.

5. Core Use Case

A repository contains a `oneconfig.yml` file. The developer runs `oneconfig init` or `oneconfig up`, and the tool installs dependencies, configures env vars, starts services, and verifies the project is ready.

6. Goals

- Reduce setup time from hours to minutes.
- Support common project stacks.
- Make repository onboarding self-documenting.
- Provide a delightful first-run experience.
- Encourage open-source adoption through simplicity.

7. Non-Goals

- Replacing Docker, Nix, or package managers.
- Managing cloud deployment.
- Building a full CI/CD platform.
- Supporting every possible language on day one.

8. Functional Requirements

8.1 Project Discovery

- FR-1 The system shall detect a repository's OneConfig file at the project root.
- FR-2 The system shall validate the config schema before executing setup steps.
- FR-3 The system shall show a clear error if the config is missing or invalid.

8.2 Environment Setup

- FR-4 The system shall install required runtime versions.
- FR-5 The system shall install dependencies for supported package managers.

- FR-6 The system shall create or update local environment variables from config.
- FR-7 The system shall support startup commands for app services and support services.

8.3 Startup Orchestration

- FR-8 The system shall execute setup steps in the correct order.
- FR-9 The system shall support parallel startup where safe.
- FR-10 The system shall wait for health checks before marking the environment ready.
- FR-11 The system shall expose logs for each startup step.

8.4 Developer Experience

- FR-12 The system shall provide a single command to bring up the project.
- FR-13 The system shall provide a single command to tear down the environment.
- FR-14 The system shall provide a status command showing what is running.
- FR-15 The system shall provide helpful troubleshooting output when setup fails.

8.5 Templates

- FR-16 The system shall provide starter templates for common stacks.
- FR-17 The system shall allow users to generate a OneConfig file from an existing project.
- FR-18 The system shall allow maintainers to customize setup instructions without writing scripts manually.

9. Supported Scope for v1

- JavaScript/TypeScript projects.
- Python projects.
- Mixed frontend/backend projects.
- Projects using environment files and basic local services.

10. Data Model

10.1 OneConfig File

- project_name
- runtimes
- package_managers
- env_vars
- services
- setup_steps
- health_checks
- post_start_commands

10.2 Service Definition

- name
- type
- start_command
- port
- health_check
- depends_on

11. API / CLI Outline

Commands

- ``oneconfig init``
- ``oneconfig up``
- ``oneconfig down``
- ``oneconfig status``
- ``oneconfig doctor``
- ``oneconfig generate``

Behavior

- ``init`` creates or validates a config.
- ``up`` installs and starts the environment.

- ``down`` stops local services.
- ``status`` reports current state.
- ``doctor`` checks missing tools and config issues.
- ``generate`` scans an existing repo and creates a draft config.

12. Success Criteria

- A new contributor can get a project running with one command.
- The tool supports at least three common real-world repos end to end.
- Setup failures are explained clearly.
- The generated config is accurate enough to reduce manual README setup.

13. Usability Principles

- One command should do the common case.
- The config should stay human-readable.
- Errors should explain the fix, not just the failure.
- Defaults should work for most projects without advanced customization.

14. Risks

- Too many supported stacks may create complexity.
- Different projects may need different setup semantics.
- Users may expect full environment virtualization when the product only handles local setup.
- Overengineering the config format may reduce adoption.

15. Future Enhancements

- Docker integration.
- Remote environment support.
- GitHub repo detection and auto-generation.
- Editor integration.
- Team-shared setup profiles.